Báo cáo bài tập E1–E3

Học sâu và ứng dụng trong thị giác máy tính (CO5085) – Học kỳ 261 (2026–2027)

Sinh viên: Phan Ngọc Trâm – MSSV 2570790 (làm cá nhân)

Repository: https://github.com/tramphanngoc174/co5085-exercises

Bài	Hạn nộp	Trạng thái trong bản báo cáo này
E1 – Softmax / MLP / CNN	02/11/2026	Đã có kết quả (mục 1)
E2 – MSA + tokenization	02/11/2026	Đã có kết quả (mục 2)
E3 – LSTM / GRU	09/11/2026	Đã có kết quả (mục 3)

Báo cáo này là một file PDF chung, được cập nhật dần theo từng hạn nộp.

0. Dữ liệu và thiết lập chung

Tập dữ liệu: Fashion-MNIST (ảnh xám 28×28, 10 lớp quần áo/giày dép), dùng chung cho E1–E3. Lý do chọn: tập nhỏ, chạy nhanh; khó hơn MNIST nên các mô hình phân biệt được với nhau, và các lớp dễ nhầm (áo) cho phép phân tích lỗi có ý nghĩa.

Chia dữ liệu: 60.000 ảnh train gốc được chia ngẫu nhiên với seed 42 thành **54.000 train** và **6.000 validation**; **10.000 ảnh test** gốc giữ nguyên. Validation dùng để chọn epoch; test chỉ dùng để báo cáo kết quả cuối. Chỉ số chia được lưu trong splits/fashion_mnist_val6000_seed42.json để E2, E3 dùng đúng cùng split. Tỷ lệ chia là lựa chọn của sinh viên (đề không quy định).

Tiền xử lý: chuẩn hóa theo mean 0,2860 và std 0,3530; không dùng data augmentation.

Môi trường: PyTorch, chạy trên Google Colab (GPU CUDA). Seed, cấu hình và log của từng lần chạy được lưu trong results/E1/ (config.json, log.txt).

1. E1 – Phân loại ảnh: Softmax, MLP, CNN

1.1 Kiến trúc

Mô hình	Cấu trúc	Số tham số
Softmax	Flatten (784) → Linear (784→10); softmax nằm trong hàm loss	7.850
MLP	Flatten (784) → Linear 512 + ReLU → Linear 256 + ReLU → Linear 10	535.818
CNN	2 khối [Conv 3×3 + BatchNorm + ReLU + MaxPool 2×2] với 32 và 64 kênh; Flatten (64·7·7 = 3136) → Linear 128 + ReLU + Dropout 0,3 → Linear 10	421.834

Cả ba mô hình trả về logit; softmax được tính bên trong CrossEntropyLoss. Softmax và MLP nhận đầu vào là vector flatten (đúng yêu cầu của đề); chỉ CNN khai thác cấu trúc không gian của ảnh.

1.2 Thiết lập huấn luyện

- Vòng lặp huấn luyện tự viết: zero_grad() → forward → loss → backward() → optimizer.step() (file common/train_utils.py).
- Optimizer Adam, learning rate 0,001, batch size 128, 15 epoch, loss Cross-Entropy; cả ba mô hình dùng cùng cấu hình.
- Mỗi epoch đánh giá trên validation; trọng số của epoch có **validation accuracy cao nhất** được giữ lại và dùng để đo test accuracy một lần duy nhất.

- Seed 42 được đặt lại trước khi huấn luyện từng mô hình. Mỗi mô hình chỉ được huấn luyện **một lần**.

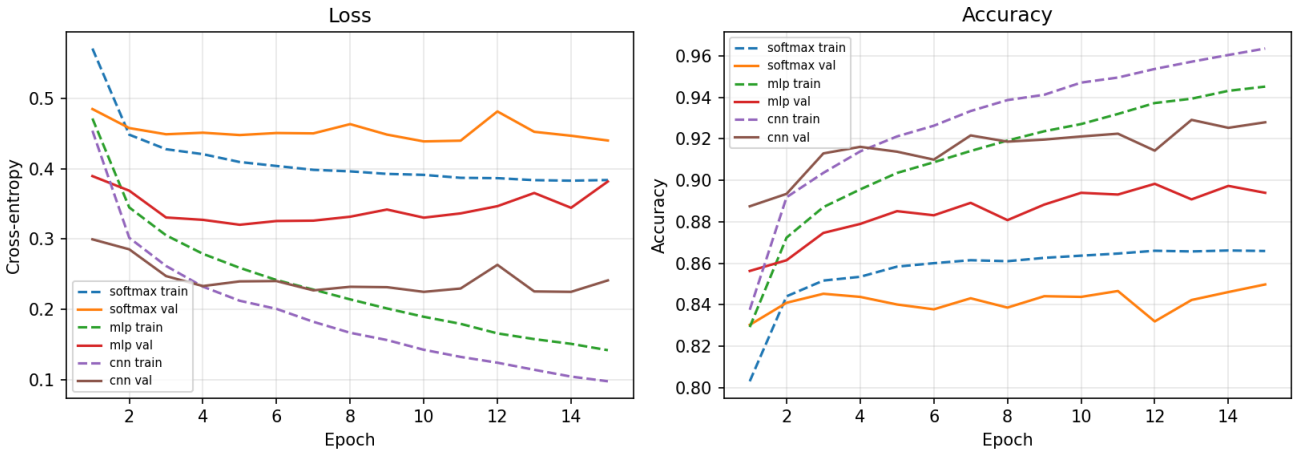
1.3 Kết quả so sánh

Mô hình	Tham số	Best epoch	Train acc (tại best epoch)	Val acc	Test acc	Giây/epoch
Softmax	7.850	15	0,8660	0,8498	0,8442	12,4
MLP	535.818	12	0,9373	0,8983	0,8896	12,8
CNN	421.834	13	0,9572	0,9292	0,9244	14,6

Nguồn: results/E1/summary.md và results/E1/log.txt (một lần chạy, seed 42, 15 epoch).

- CNN đạt test accuracy cao nhất (0,9244), hơn MLP 3,5 điểm phần trăm (0,8896) và hơn Softmax 8,0 điểm (0,8442). MLP hơn Softmax 4,5 điểm.
- CNN có **ít tham số hơn MLP** (421.834 so với 535.818, ít hơn khoảng 21%) nhưng chính xác hơn; trong thí nghiệm này, số tham số không quyết định chất lượng mà cách mô hình khai thác cấu trúc ảnh quan trọng hơn.
- Test accuracy thấp hơn validation accuracy từ 0,5 đến 0,9 điểm ở cả ba mô hình, điều này là bình thường vì epoch được chọn theo validation.
- Thời gian mỗi epoch chênh lệch nhỏ (12,4–14,6 giây); do đó thời gian không phải yếu tố phân biệt trong thí nghiệm này.

1.4 Đường cong học



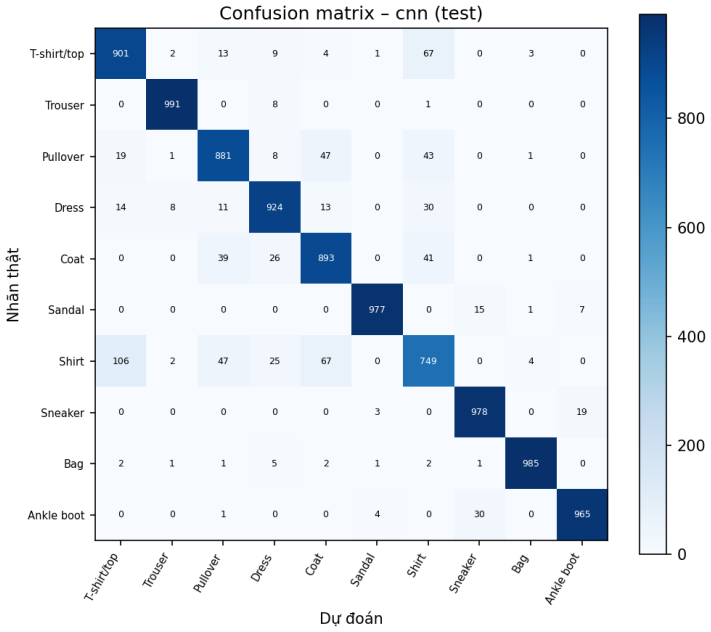
Hình 1. Loss và accuracy theo epoch trên tập train (nét đứt) và validation (nét liền) của ba mô hình.

- **Softmax**: train accuracy chỉ đạt khoảng 0,866 và train loss giảm rất chậm sau vài epoch đầu; validation accuracy dao động trong khoảng 0,83–0,85. Mô hình tuyến tính bị giới hạn bởi capacity thấp, chưa phải do thiếu epoch.
- **MLP**: train loss giảm liên tục (đến 0,1419) nhưng validation loss thấp nhất ở epoch 5 (0,3204) rồi tăng lên 0,3820 ở epoch 15, trong khi train accuracy (0,9452) vượt xa validation accuracy (0,8940). Đây là dấu hiệu overfitting.
- **CNN**: validation loss thấp nhất khoảng 0,2249 (epoch 10 và 14) và tăng nhẹ lên 0,2412 ở epoch 15, train accuracy 0,9636. Overfitting có xuất hiện nhưng nhẹ hơn MLP (khoảng cách train–validation tại best epoch là 2,8 điểm so với 3,9 điểm của MLP).
- Validation accuracy của CNN dao động giữa các epoch (từ 0,9143 đến 0,9292 trong epoch 11–15), nên việc chọn đúng một epoch tốt nhất có yếu tố ngẫu nhiên.

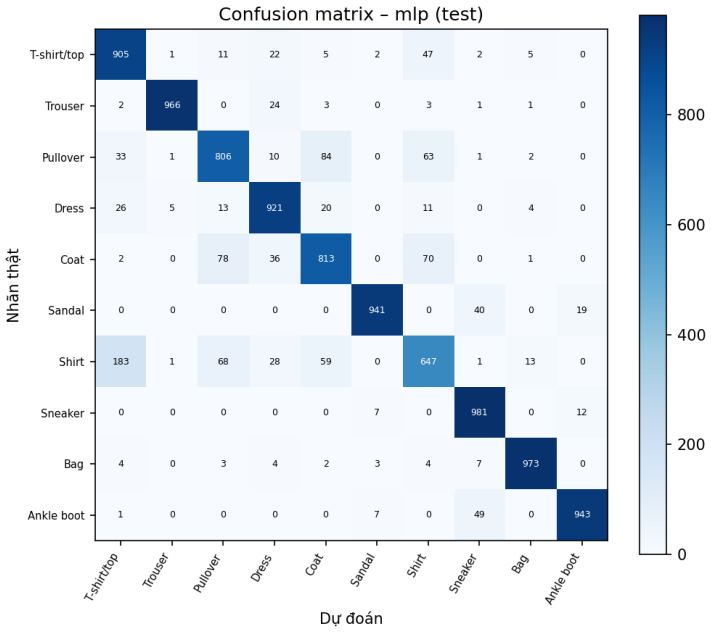
1.5 Phân tích lỗi

Số mẫu đúng trên 1.000 ảnh test của mỗi lớp (đường chéo của confusion matrix), nên cũng bằng recall tính theo phần nghìn:

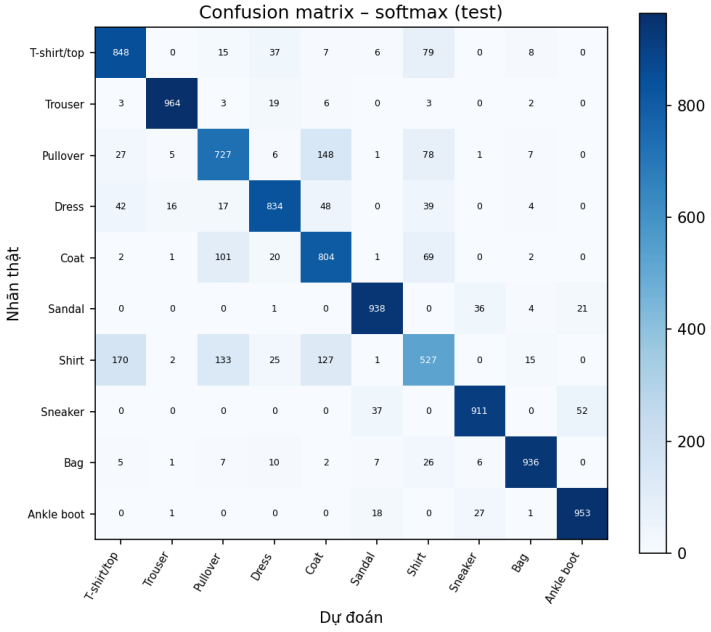
Lớp	Softmax	MLP	CNN
T-shirt/top	848	905	901
Trouser	964	966	991
Pullover	727	806	881
Dress	834	921	924
Coat	804	813	893
Sandal	938	941	977
Shirt	527	647	749
Sneaker	911	981	978
Bag	936	973	985
Ankle boot	953	943	965
Tổng (đúng/10.000)	8442	8896	9244



Hình 2. Confusion matrix của CNN trên tập test.

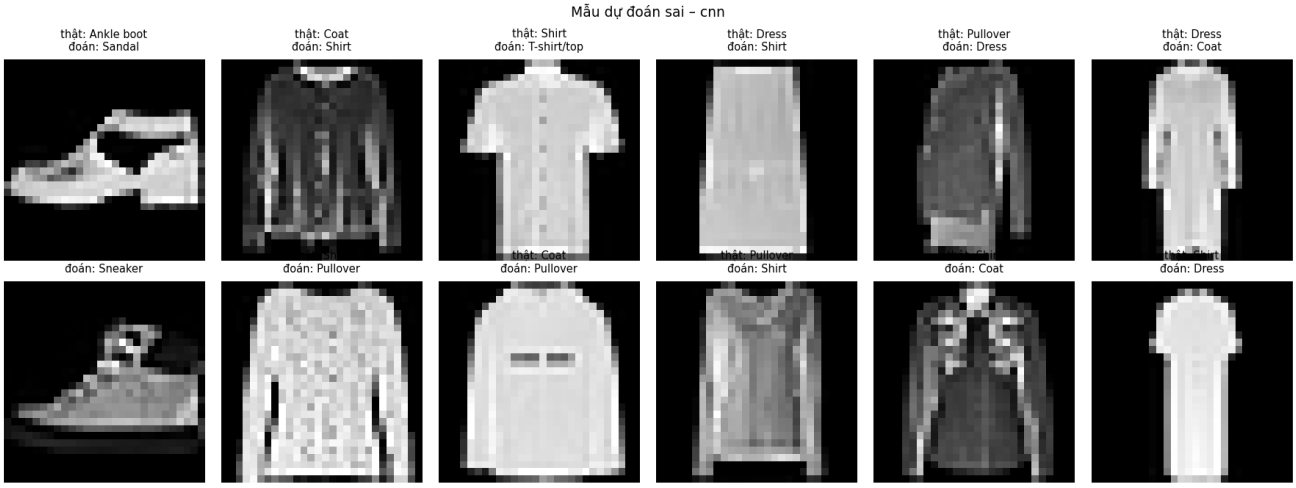


Hình 3. Confusion matrix của MLP trên tập test.

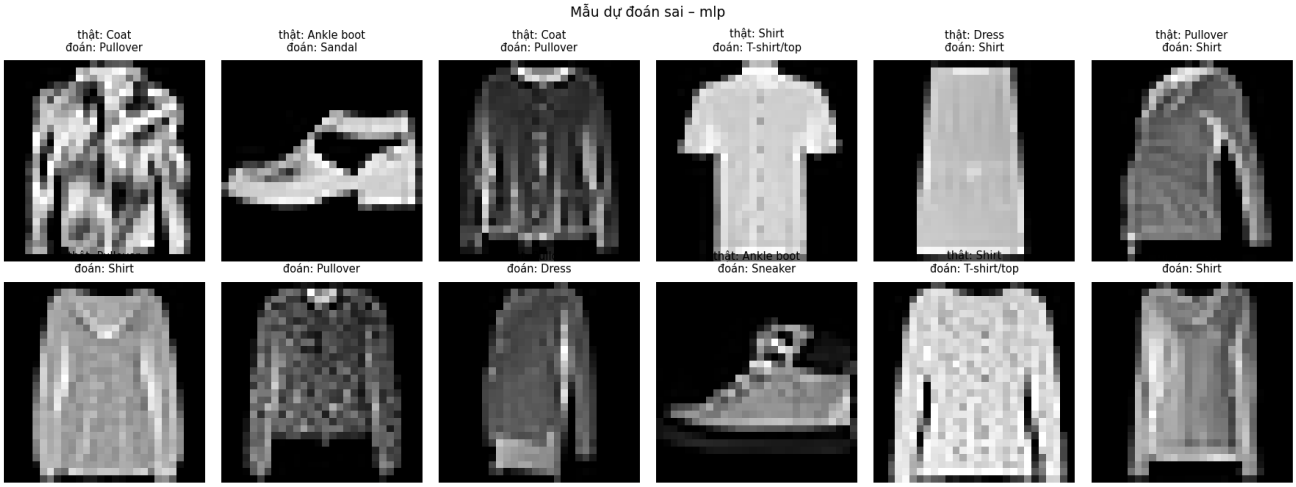


Hình 4. Confusion matrix của Softmax trên tập test.

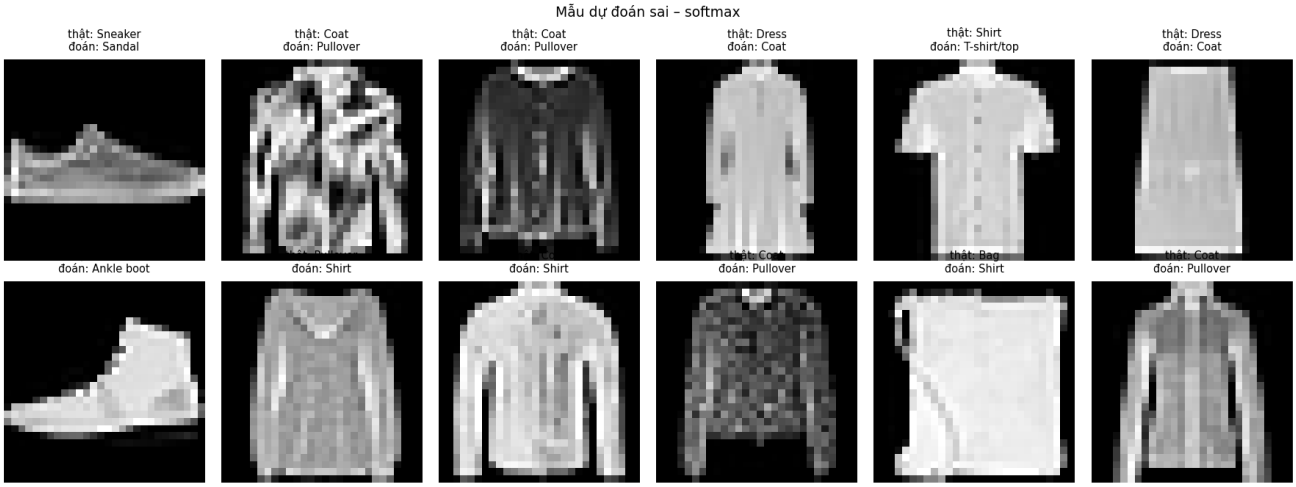
- **Lớp khó nhất ở cả ba mô hình là Shirt**: đúng 527 (Softmax), 647 (MLP), 749 (CNN) trên 1.000 ảnh. Shirt bị nhầm nhiều nhất sang T-shirt/top (170, 183 và 106 ảnh), rồi sang Pullover và Coat.
- Các lỗi tập trung vào nhóm áo: T-shirt/top, Shirt, Pullover, Coat, Dress. Ví dụ Pullover bị nhầm sang Coat 148 (Softmax), 84 (MLP), 47 (CNN) ảnh.
- Nhóm giày dép nhầm lẫn lẫn nhau: Ankle boot bị đoán là Sneaker 27 (Softmax), 49 (MLP), 30 (CNN) ảnh; Sneaker bị đoán là Ankle boot 52, 12, 19 ảnh.
- Trouser, Bag, Sneaker, Sandal và Ankle boot được nhận dạng tốt (từ 911 đến 991 trên 1.000 ảnh) ở mọi mô hình, nên phần lớn sự khác biệt giữa các mô hình nằm ở nhóm áo.
- So với Softmax, CNN tăng recall của Shirt thêm 222 ảnh trên 1.000 (từ 527 lên 749) và so với MLP thêm 102 ảnh (từ 647 lên 749).



Hình 5. Mười hai mẫu test đầu tiên bị CNN dự đoán sai (nhãn thật và nhãn dự đoán ghi trên mỗi ảnh).



Hình 6. Mười hai mẫu test đầu tiên bị MLP dự đoán sai.



Hình 7. Mười hai mẫu test đầu tiên bị Softmax dự đoán sai.

Lưu ý: các hình 5–7 là 12 mẫu sai đầu tiên theo thứ tự trong tập test, không phải lấy ngẫu nhiên, nên một số ảnh xuất hiện ở nhiều mô hình. Hai dòng tiêu đề ở hàng dưới của các hình bị chồng lên ảnh phía trên (lỗi hiển thị nhỏ, không ảnh hưởng nhãn).

- Một số mẫu khó phân biệt kể cả khi nhìn bằng mắt, ví dụ áo sơ mi ngắn tay có cúc (nhãn Shirt) bị cả ba mô hình đoán là T-shirt/top, hoặc đôi giày có quai, gót cao (nhãn Ankle boot) bị CNN và MLP đoán là Sandal.

- Ảnh 28×28 xám bỏ mất nhiều chi tiết (cổ áo, cúc, độ dài tay), nên ranh giới giữa các lớp áo vốn mờ.

1.6 Nhận xét và hạn chế

- Thứ hạng CNN > MLP > Softmax nhất quán trên cả validation và test. Softmax bị giới hạn bởi capacity (underfitting); MLP có đủ capacity nhưng overfit sau khoảng epoch 5; CNN đạt kết quả tốt nhất với ít tham số hơn MLP.
- Giải thích thường gặp cho việc CNN tốt hơn là convolution dùng chung trọng số và bắt đặc trưng cục bộ (cạnh, hình dạng) bất kể vị trí trong ảnh, trong khi MLP và Softmax xem mỗi pixel là một đầu vào độc lập. Báo cáo này không có thí nghiệm riêng (ablation) để kiểm chứng giải thích đó.
- Hạn chế: mỗi mô hình chỉ chạy một lần với một seed nên chưa đo được độ dao động giữa các lần chạy; chưa tinh chỉnh siêu tham số, chưa dùng data augmentation hay dừng sớm; CNN có thêm BatchNorm và Dropout còn MLP thì không, nên phần chênh lệch giữa hai mô hình không chỉ do convolution.

2. E2 – Transformer: MSA và tokenization

2.1 Thiết lập chung

- Bộ phân loại: Tokenizer → 2 khối encoder [LayerNorm → MSA → cộng tắt, LayerNorm → FFN → cộng tắt] → LayerNorm → Linear 10 trên token CLS. Kích thước ẩn 64, 4 head, FFN có hệ số mở rộng 2 (GELU). Mọi tokenizer đều thêm token CLS và positional embedding học được.
- Dữ liệu, split, seed 42 và vòng lặp huấn luyện tự viết giống E1; Adam, learning rate 0,001, batch 128, loss Cross-Entropy, chọn epoch theo validation accuracy. Khác E1: **10 epoch** (thay vì 15) để tiết kiệm thời gian chạy.
- Có 3 cách tokenize × 2 bản MSA = 6 lần huấn luyện, mỗi lần một seed. Các lần chạy dùng chung cấu hình (xem results/E2/config.json).

2.2 MSA tự viết và MSA PyTorch

MSA tự viết (E2/msa.py, lớp CustomMSA) chỉ dùng Linear, nhân ma trận, softmax và ghép head: ba lớp Linear tạo Q, K, V; tách thành 4 head (mỗi head 16 chiều); điểm chú ý = $Q \cdot K^T$ chia căn bậc hai của 16; softmax theo chiều token; nhân với V; ghép các head lại và qua một lớp Linear đầu ra. Bản này không dùng nn.MultiheadAttention hay nn.TransformerEncoder. **MSA PyTorch** là nn.MultiheadAttention (TorchMSA) với cùng giao diện.

Kiểm tra tính đúng: chép trọng số của bản tự viết sang bản PyTorch rồi so đầu ra trên một batch ngẫu nhiên (3 mẫu × 10 token × 64 chiều). Sai khác tuyệt đối lớn nhất là **$1{,}19 \times 10^{-7}$** (ngưỡng kiểm tra 10^{-5}). Cả hai bản có cùng **16.640** tham số mỗi khối MSA. (Kiểm tra này nằm trong E2/smoke_test.py.) Khi huấn luyện, hai bản khởi tạo trọng số theo cách khác nhau, nên kết quả huấn luyện của chúng không giống hệt nhau dù cùng seed.

2.3 Ba cách tokenize

Tokenizer	Cách lấy token	Số token (kèm CLS)
patch	Chia ảnh thành lưới ô 4×4 không chồng lấn (49 ô); mỗi ô 16 giá trị qua Linear 16→64	50
row	Mỗi hàng của ảnh (28 giá trị) là một token (28 token); qua Linear 28→64	29
cnnstem	CNN 2 khối [Conv 3×3 + BatchNorm + ReLU + MaxPool] (32 và 64 kênh) cho feature map 7×7; mỗi vị trí là một token 64 chiều (49 token), qua Linear 64→64	50

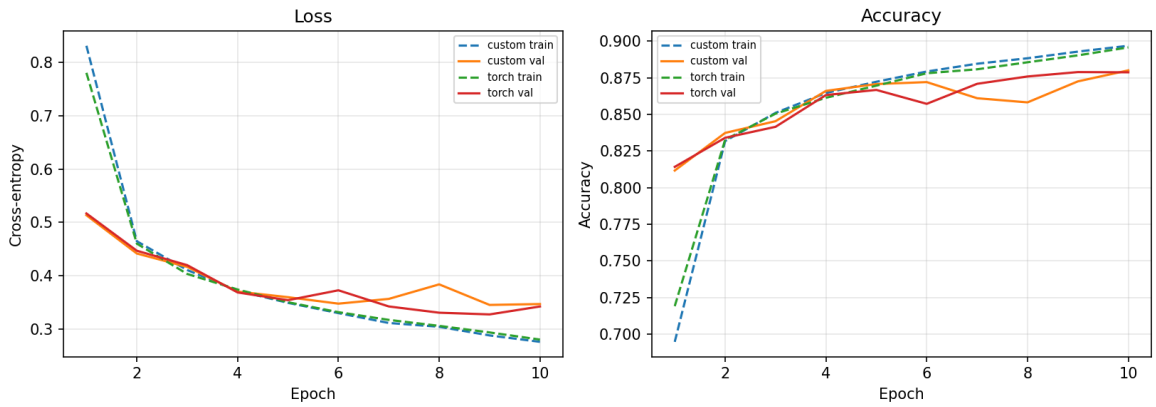
2.4 Kết quả

Tokenizer	MSA	Tham số	Best epoch	Val acc	Test acc	Giây/epoch	Tổng train (s)
patch	tự viết	72.074	10	0,8800	0,8753	15,5	155
patch	PyTorch	72.074	9	0,8788	0,8736	15,7	157
row	tự viết	71.498	9	0,8888	0,8816	14,8	148
row	PyTorch	71.498	6	0,8878	0,8784	14,8	148
cnnstem	tự viết	94.154	10	0,9138	0,9077	17,7	177
cnnstem	PyTorch	94.154	8	0,9060	0,8986	18,1	181

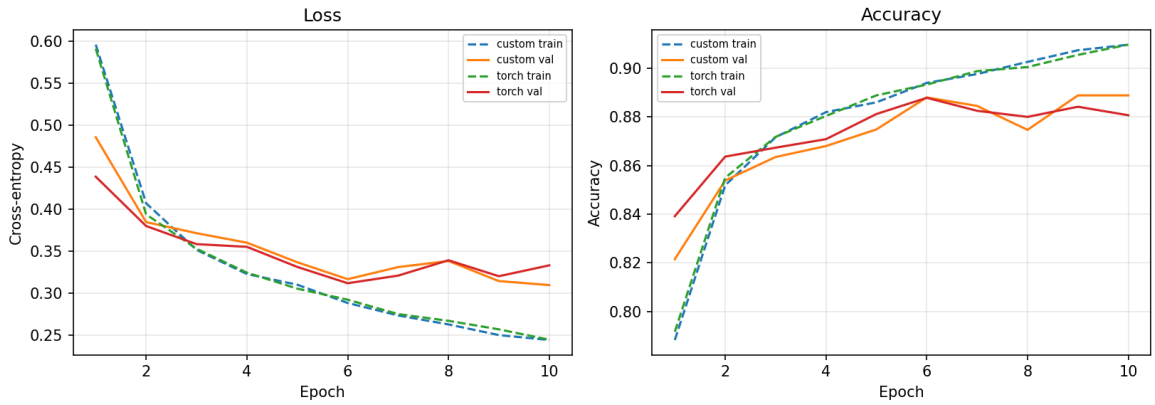
Nguồn: results/E2/summary.md và log.txt (10 epoch, seed 42, GPU CUDA).

2.5 So sánh (i): MSA tự viết và MSA PyTorch (cùng cách tokenize)

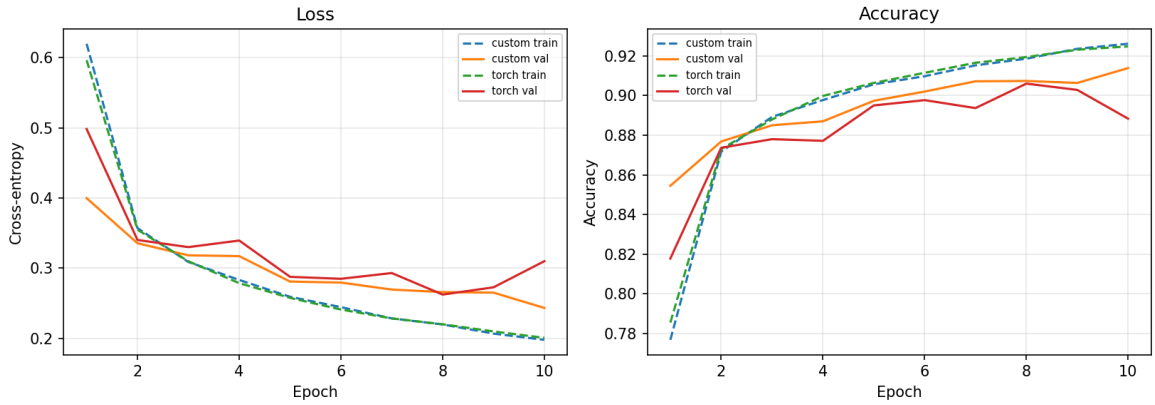
Tokenizer	Test acc tự viết	Test acc PyTorch	Chênh lệch (tự viết − PyTorch)
patch	0,8753	0,8736	+0,17 điểm
row	0,8816	0,8784	+0,32 điểm
cnnstem	0,9077	0,8986	+0,91 điểm



Hình 8. Tokenizer patch: loss và accuracy theo epoch của MSA tự viết (custom) và MSA PyTorch (torch).



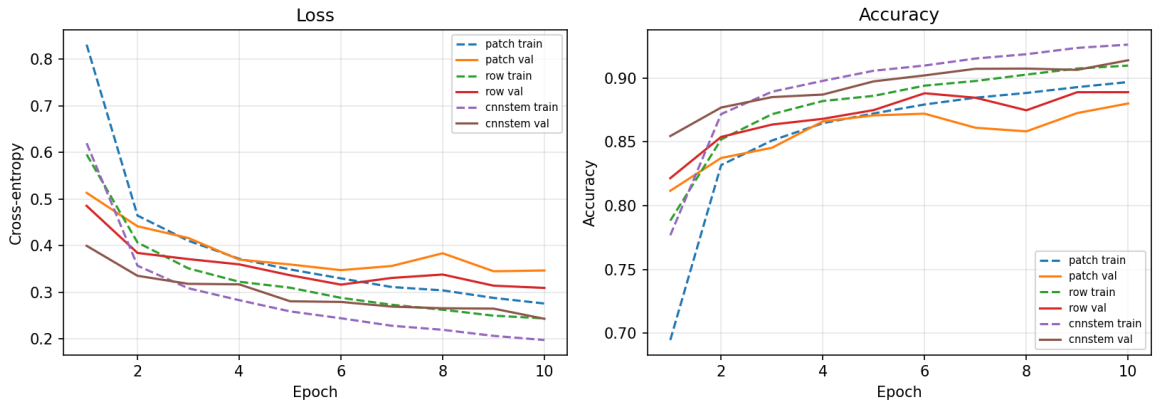
Hình 9. Tokenizer row: custom và torch.



Hình 10. Tokenizer cnnstem: custom và torch.

- Hai bản cho kết quả rất gần nhau: chênh lệch test accuracy là 0,17 điểm (patch), 0,32 điểm (row) và 0,91 điểm (cnnstem), và đường train loss/accuracy của hai bản gần như chồng lên nhau ở cả ba tokenizer (train accuracy epoch 10: 0,8968 và 0,8957 với patch; 0,9097 và 0,9097 với row; 0,9261 và 0,9248 với cnnstem). Điều này phù hợp với việc MSA tự viết tính cùng hàm với bản PyTorch.
- Bản tự viết có test accuracy nhỉnh hơn ở cả ba tokenizer, nhưng chưa thể kết luận bản nào tốt hơn: mỗi cấu hình chỉ chạy một lần, và validation accuracy dao động giữa các epoch (ví dụ cnnstem–PyTorch giảm từ 0,9028 ở epoch 9 xuống 0,8883 ở epoch 10, tức 1,45 điểm, lớn hơn khoảng chênh 0,91 điểm giữa hai bản). Epoch tốt nhất cũng khác nhau giữa hai bản (ví dụ 6 so với 9 với row).
- Thời gian huấn luyện gần như bằng nhau (155 và 157 s; 148 và 148 s; 177 và 181 s), nên trong thiết lập này bản PyTorch không nhanh hơn rõ rệt. Chuỗi chỉ có 29–50 token và mô hình nhỏ, nên thời gian có thể bị chi phối bởi các phần khác (nạp dữ liệu, các lớp còn lại); báo cáo không có phép đo riêng để khẳng định.

2.6 So sánh (ii): các cách tokenize (dùng MSA tự viết)



Hình 11. So sánh ba tokenizer với MSA tự viết (nét đứt: train, nét liền: validation).

- **cnnstem cho kết quả tốt nhất**: test accuracy 0,9077, cao hơn row (0,8816) 2,6 điểm và cao hơn patch (0,8753) 3,2 điểm; đường validation của cnnstem nằm trên hai tokenizer còn lại ở hầu hết các epoch. Đổi lại cnnstem có nhiều tham số hơn (94.154 so với khoảng 72.000, nhiều hơn khoảng 31%) và chậm hơn (17,7 s/epoch so với 14,8 s của row và 15,5 s của patch).
- **row** (0,8816) nhỉnh hơn **patch** (0,8753) 0,6 điểm, nhưng chênh lệch nhỏ và validation accuracy của patch dao động nhiều (0,8720 ở epoch 6, rồi 0,8610 và 0,8582 ở epoch 7–8), nên chưa nên kết luận chắc về thứ tự của hai tokenizer này. Row dùng ít token nhất (29) và nhanh nhất (14,8 s/epoch).
- patch học chậm hơn ở những epoch đầu (train accuracy epoch 1 là 0,6948 so với 0,7883 của row và 0,7769 của cnnstem).
- Ở cả ba cấu hình, train accuracy vẫn còn tăng ở epoch 10 và khoảng cách train–validation nhỏ (1,2 đến 2,1 điểm), nên mô hình có thể chưa hội tụ hết sau 10 epoch; báo cáo chưa thử huấn luyện lâu hơn.
- Tham khảo (không cùng điều kiện): CNN của E1 đạt test accuracy 0,9244 với 15 epoch và 421.834 tham số; các Transformer nhỏ ở E2 (10 epoch, 71.000–94.000 tham số) đạt 0,8736–0,9077.

2.7 Nhận xét và hạn chế

- MSA tự viết đúng về số học (sai khác $1{,}19\times10^{-7}$ so với PyTorch khi cùng trọng số) và cho kết quả huấn luyện gần bản PyTorch ở cả ba cách tokenize.
- Trong các lần chạy này, cách lấy token ảnh hưởng nhiều hơn việc chọn bản MSA: chênh lệch giữa các tokenizer (tới 3,2 điểm) lớn hơn chênh lệch giữa hai bản MSA (tới 0,9 điểm).
- Một giải thích có thể cho ưu thế của cnnstem là tầng convolution đã rút đặc trưng cục bộ trước khi attention làm việc, trong khi patch và row đưa giá trị pixel thô qua một phép Linear. Báo cáo không có thí nghiệm riêng (ablation) để kiểm chứng; cnnstem cũng có nhiều tham số hơn nên phần cải thiện không chỉ do cách tokenize.
- Hạn chế: mỗi cấu hình chỉ chạy một lần với một seed nên chưa đo được độ dao động giữa các lần chạy; chỉ 10 epoch; chưa tinh chỉnh siêu tham số (kích thước ẩn, số tầng, số head, patch size), chưa dùng dropout, data augmentation hay lịch learning rate; chỉ thử patch 4×4 không chồng lấn và chưa thử các biến thể khác (patch chồng lấn, patch size khác, token theo kênh).

3. E3 – LSTM / GRU trên chuỗi ảnh

3.1 Thiết lập chung

- Mô hình: một tầng nn.LSTM hoặc nn.GRU (để cho phép dùng module có sẵn) với 128 đơn vị ẩn, đầu vào là chuỗi do ảnh tạo ra; phân loại bằng trạng thái ẩn ở bước cuối qua một lớp Linear 10 lớp (file E3/models.py).
- Dữ liệu, split, seed 42 và vòng lặp huấn luyện tự viết giống E1; Adam, learning rate 0,001, batch 128, Cross-Entropy, **10 epoch**, chọn epoch theo validation accuracy. Mỗi cấu hình chỉ chạy một lần.
- Baseline không hồi quy: **MLP và CNN của E1**, nạp từ checkpoint đã lưu (results/E1/mlp, results/E1/cnn), cùng split và cùng tập test. Hai baseline này được huấn luyện 15 epoch ở E1 (chọn epoch 12 và 13), còn LSTM/GRU chỉ 10 epoch.

3.2 Ba cách biến ảnh thành chuỗi

Tên	Cách tạo chuỗi	Chuỗi (bước × giá trị)
row	Mỗi hàng của ảnh là một bước thời gian; đọc từ hàng trên xuống hàng dưới	28 × 28
col	Mỗi cột của ảnh là một bước thời gian; đọc từ cột trái sang cột phải	28 × 28
patch	Ảnh chia thành các ô 4×4 không chồng lấn; mỗi ô (16 giá trị) là một bước, quét theo hàng từ trái sang phải rồi từ trên xuống dưới	49 × 16

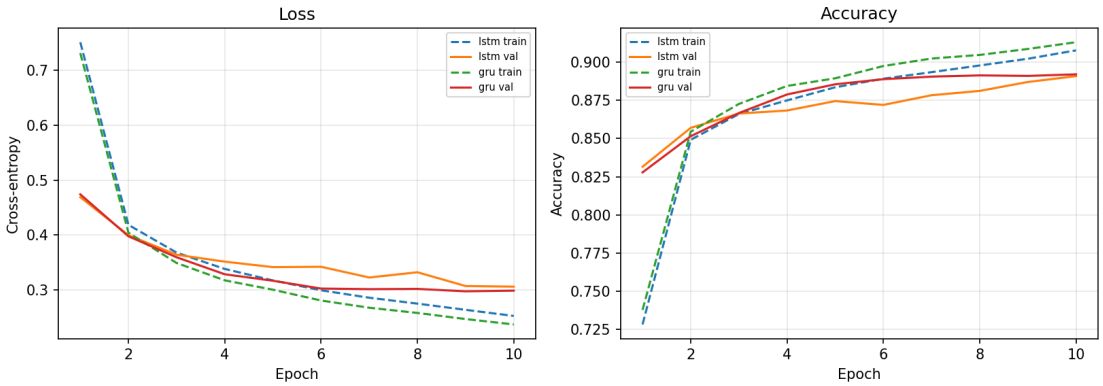
3.3 Kết quả

Mô hình	Chuỗi	Tham số	Best epoch	Val acc	Test acc	Giây/epoch
MLP (E1)	–	535.818	–	0,8983	0,8896	–
CNN (E1)	–	421.834	–	0,9292	0,9244	–
LSTM	row	82.186	10	0,8908	0,8838	13,0
GRU	row	61.962	10	0,8920	0,8904	12,5
LSTM	col	82.186	9	0,8805	0,8783	13,1
GRU	col	61.962	8	0,8865	0,8822	12,5
LSTM	patch	76.042	10	0,8695	0,8631	13,8
GRU	patch	57.354	9	0,8747	0,8679	12,6

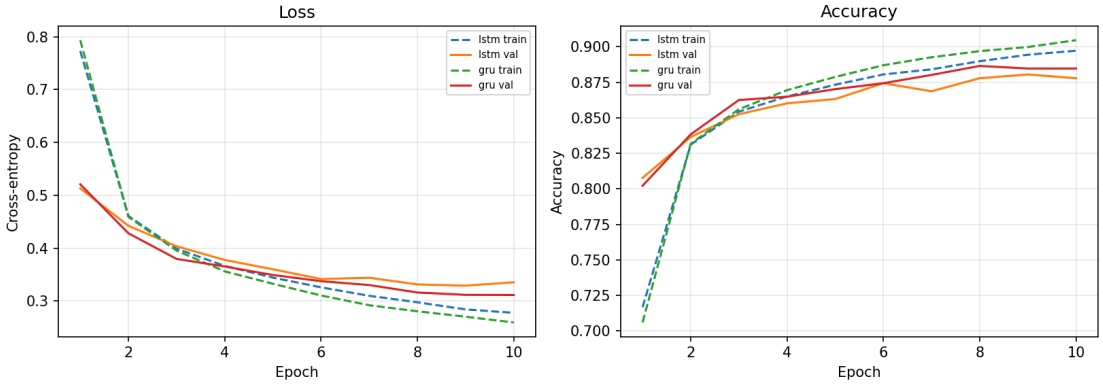
Nguồn: results/E3/summary.md và log.txt (seed 42, GPU CUDA).

3.4 So sánh LSTM và GRU (cùng cách tạo chuỗi)

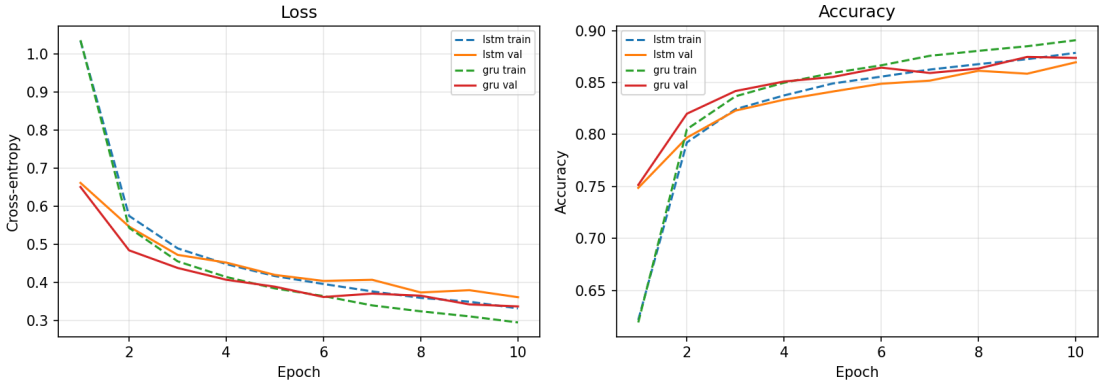
Chuỗi	Test acc LSTM	Test acc GRU	Chênh lệch (GRU − LSTM)	Tham số LSTM / GRU
row	0,8838	0,8904	+0,66 điểm	82.186 / 61.962
col	0,8783	0,8822	+0,39 điểm	82.186 / 61.962
patch	0,8631	0,8679	+0,48 điểm	76.042 / 57.354



Hình 12. Chuỗi row: loss và accuracy của LSTM và GRU.



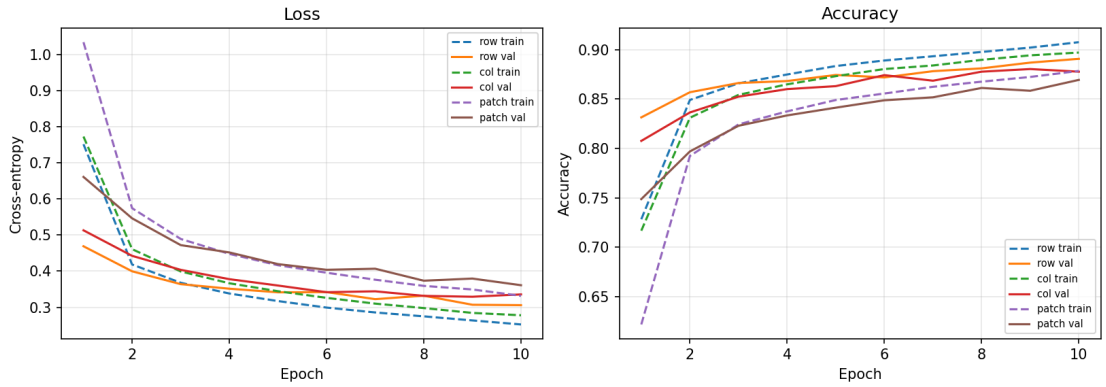
Hình 13. Chuỗi col: LSTM và GRU.



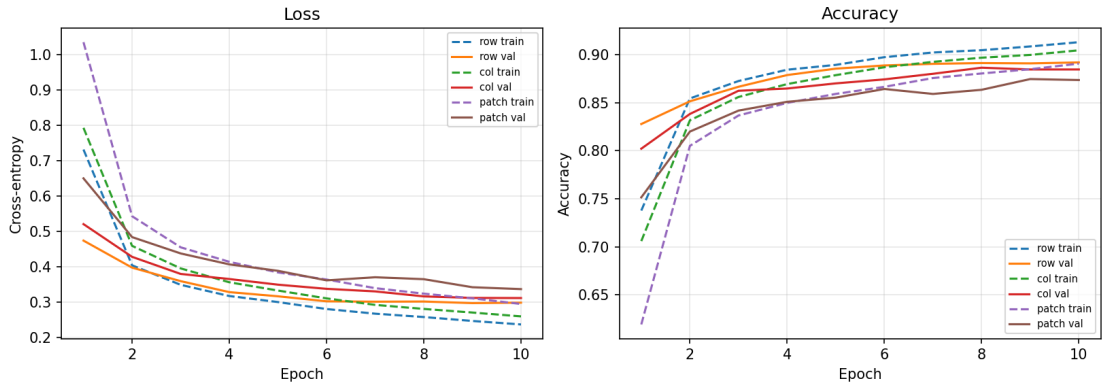
Hình 14. Chuỗi patch: LSTM và GRU.

- GRU có test accuracy cao hơn LSTM ở cả ba cách tạo chuỗi, nhưng chỉ từ 0,39 đến 0,66 điểm; với một seed và validation accuracy còn dao động, chênh lệch này nhỏ nên chưa đủ để kết luận GRU tốt hơn.
- GRU có ít tham số hơn khoảng 25% (61.962 so với 82.186; 57.354 so với 76.042) và nhỉnh hơn về tốc độ (12,5–12,6 s/epoch so với 13,0–13,8 s). Do đó GRU đạt kết quả tương đương hoặc nhỉnh hơn với chi phí thấp hơn trong thiết lập này.
- Trong cả ba hình, train loss của GRU thấp hơn LSTM ở cùng epoch (ví dụ chuỗi row: 0,2368 so với 0,2524 ở epoch 10), tức là GRU khớp dữ liệu train nhanh hơn một chút.

3.5 So sánh các cách tạo chuỗi



Hình 15. LSTM: so sánh ba cách tạo chuỗi.



Hình 16. GRU: so sánh ba cách tạo chuỗi.

- Thứ tự giống nhau cho cả hai loại RNN: **row > col > patch**. Với GRU: 0,8904, 0,8822 và 0,8679; với LSTM: 0,8838, 0,8783 và 0,8631.
- Row hơn patch khoảng 2 điểm (2,25 điểm với GRU, 2,07 điểm với LSTM) và hơn col 0,6–0,8 điểm (0,82 điểm với GRU, 0,55 điểm với LSTM). Chênh lệch giữa row và col nhỏ, còn chênh lệch với patch rõ hơn và nhất quán ở cả hai RNN.
- Chuỗi patch học chậm hơn ở những epoch đầu (train accuracy epoch 1 khoảng 0,62 so với khoảng 0,73 của row) và có chuỗi dài hơn (49 bước so với 28). Cách tạo chuỗi patch cũng khác về số giá trị mỗi bước (16 so với 28), nên báo cáo không tách được ảnh hưởng của độ dài chuỗi với ảnh hưởng của cách cắt.
- Cả ba cách tạo chuỗi có best epoch ở 8–10 trong 10 epoch và đường validation vẫn đang đi lên hoặc đi ngang nhẹ, nên mô hình có thể chưa hội tụ hết.

3.6 So sánh với baseline MLP và CNN của E1

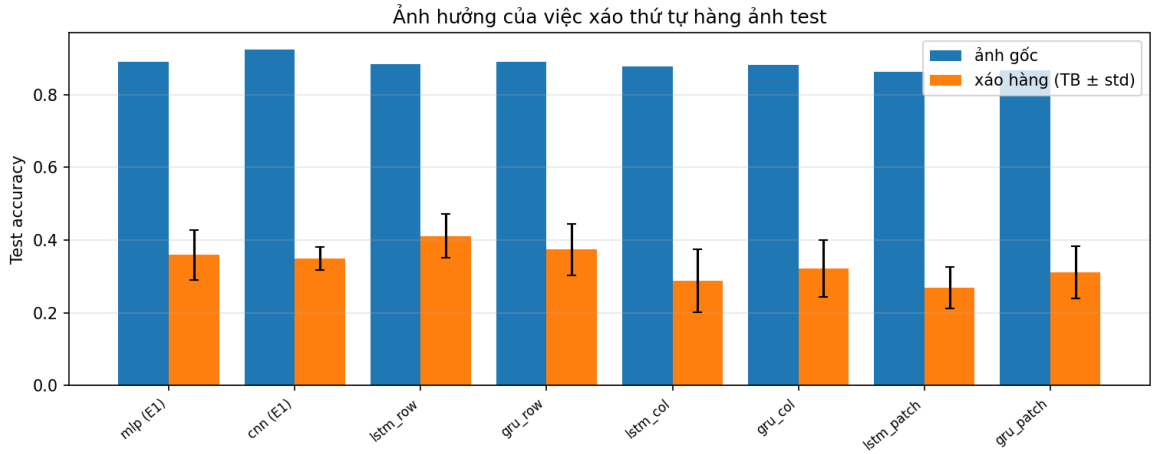
- GRU–row (0,8904) **ngang MLP** (0,8896; chênh 0,08 điểm) nhưng chỉ dùng 61.962 tham số, ít hơn khoảng 8,6 lần (535.818).
- GRU–row **thấp hơn CNN** 3,4 điểm (0,8904 so với 0,9244) và dùng ít tham số hơn khoảng 6,8 lần (421.834).
- LSTM/GRU với chuỗi col và patch đều thấp hơn MLP (0,8631–0,8822 so với 0,8896) và thấp hơn CNN.
- Lưu ý khi so sánh: baseline E1 được huấn luyện 15 epoch còn RNN 10 epoch, và số tham số khác nhau nhiều; vì vậy bảng này cho thấy vị trí tương đối của các mô hình trong điều kiện đã chạy, không phải so sánh công bằng tuyệt đối.

3.7 Thí nghiệm xáo trộn thứ tự hàng ảnh

Mục đích: xem mỗi mô hình phụ thuộc vào thứ tự hàng của ảnh đến mức nào. **Cách làm:** các mô hình đã huấn luyện trên ảnh bình thường (không huấn luyện lại) được đánh giá trên tập test mà các hàng của mỗi ảnh bị đổi thứ tự. Dùng 5 hoán vị ngẫu nhiên cố định (cùng 5 hoán vị cho mọi mô hình), báo cáo trung bình ± độ lệch chuẩn; ngoài ra thử một phép đảo ngược thứ tự hàng (lật dọc ảnh). Ngẫu nhiên đoán bừa trên 10 lớp cho accuracy khoảng 0,10.

Mô hình	Tham số	Acc gốc	Acc xáo hàng (TB ± std)	Giảm (điểm)	Acc đảo ngược	Giảm (điểm)
MLP (E1)	535.818	0,8896	0,3590 ± 0,0693	53,06	0,2225	66,71
CNN (E1)	421.834	0,9244	0,3495 ± 0,0316	57,49	0,2272	69,72
LSTM row	82.186	0,8838	0,4116 ± 0,0605	47,22	0,2498	63,40
GRU row	61.962	0,8904	0,3744 ± 0,0706	51,60	0,2105	67,99
LSTM col	82.186	0,8783	0,2881 ± 0,0871	59,02	0,2261	65,22
GRU col	61.962	0,8822	0,3230 ± 0,0782	55,92	0,1992	68,30
LSTM patch	76.042	0,8631	0,2693 ± 0,0578	59,38	0,2615	60,16
GRU patch	57.354	0,8679	0,3114 ± 0,0715	55,65	0,2194	64,85

Nguồn: results/E3/shuffle.md. Độ lệch chuẩn tính trên 5 hoán vị (ddof = 1).



Hình 17. Test accuracy trên ảnh gốc và trên ảnh bị xáo hàng (cột lỗi: độ lệch chuẩn giữa 5 hoán vị).

- **Mọi mô hình đều sụp đổ khi hàng bị xáo:** accuracy giảm từ 47 đến 59 điểm, còn 0,27–0,41 (so với khoảng 0,88–0,92 ban đầu); không mô hình nào bền với việc đổi thứ tự hàng.
- **So với MLP và CNN:** hai mô hình RNN theo hàng (LSTM row giảm 47,22 điểm, GRU row giảm 51,60 điểm) giảm ít hơn CNN (57,49) và ở mức gần MLP (53,06). Tuy nhiên độ lệch chuẩn giữa các hoán vị lớn (3 đến 9 điểm), cùng bậc với chênh lệch giữa các mô hình, nên thí nghiệm này **chưa cho thấy rõ** RNN bền hơn MLP hay CNN.
- Đảo ngược thứ tự hàng làm accuracy thấp hơn trung bình của các hoán vị ngẫu nhiên ở cả 8 mô hình (còn 0,20–0,26), tức giảm 60–70 điểm.
- **Cách diễn giải cần thận trọng:** chỉ với mô hình row, xáo hàng mới đúng là xáo thứ tự các bước thời gian. Với col và patch, xáo hàng làm thay đổi nội dung bên trong mỗi bước chứ không chỉ đổi thứ tự các bước, nên mức giảm của chúng không đo thuần túy phụ thuộc thời gian. MLP và CNN cũng gục vì các đặc trưng học được gắn với vị trí pixel. Do đó thí nghiệm cho thấy mô hình nhạy với thứ tự hàng đến đâu, nhưng không chứng minh trực tiếp RNN “bắt phụ thuộc thời gian” tốt hơn MLP/CNN.

3.8 Nhận xét và hạn chế

- RNN nhỏ (khoảng 57.000–82.000 tham số) đạt 0,863–0,890 test accuracy; tốt nhất là GRU với chuỗi row (0,8904), ngang MLP nhưng kém CNN 3,4 điểm. Cách tạo chuỗi quan trọng hơn việc chọn LSTM hay GRU: chênh lệch giữa các cách tạo chuỗi (tới 2,3 điểm) lớn hơn chênh lệch giữa LSTM và GRU (dưới 0,7 điểm).
- Hạn chế: mỗi cấu hình chỉ chạy một lần với một seed; chỉ 10 epoch và nhiều đường cong vẫn đang đi lên; chưa tinh chỉnh siêu tham số (số đơn vị ẩn, số tầng, learning rate); chỉ dùng một tầng RNN một chiều và chỉ lấy trạng thái ẩn cuối; baseline E1 huấn luyện 15 epoch trong khi RNN 10 epoch.
- Thí nghiệm xáo hàng chỉ xáo ở thời điểm test, chỉ dùng 5 hoán vị và không huấn luyện lại mô hình trên dữ liệu đã xáo, nên chưa trả lời được câu hỏi mô hình nào có thể học tốt khi thứ tự hàng bị phá vỡ.

4. Ghi nhận việc sử dụng công cụ AI

Công cụ AI Claude (Anthropic) được dùng để hỗ trợ: phân tích đề bài và lập kế hoạch nộp bài; soạn khung mã nguồn E1, E2 và E3 (các file trong common/, E1/, E2/ và E3/), tệp khung README, trang GitHub Pages; và soạn bản nháp báo cáo từ số liệu thật do sinh viên cung cấp. Sinh viên chạy thí nghiệm trên Google Colab, cung cấp log và hình kết quả, và chịu trách nhiệm kiểm tra và giải thích mã nguồn cũng như số liệu đã nộp.

5. Tái lập kết quả E1, E2 và E3

Từ thư mục gốc của repo: python E1/run_e1.py --dataset fashion_mnist --epochs 15. Kết quả nằm trong results/E1/. E2: python E2/run_e2.py --dataset fashion_mnist --epochs 10, kết quả nằm trong results/E2/. E3: python E3/run_e3.py --dataset fashion_mnist --epochs 10 (cần checkpoint E1 trong results/E1/), kết quả nằm trong results/E3/ (log, cấu hình, bảng, biểu đồ, checkpoint). Hướng dẫn môi trường và lệnh chạy chi tiết: xem README của repository.